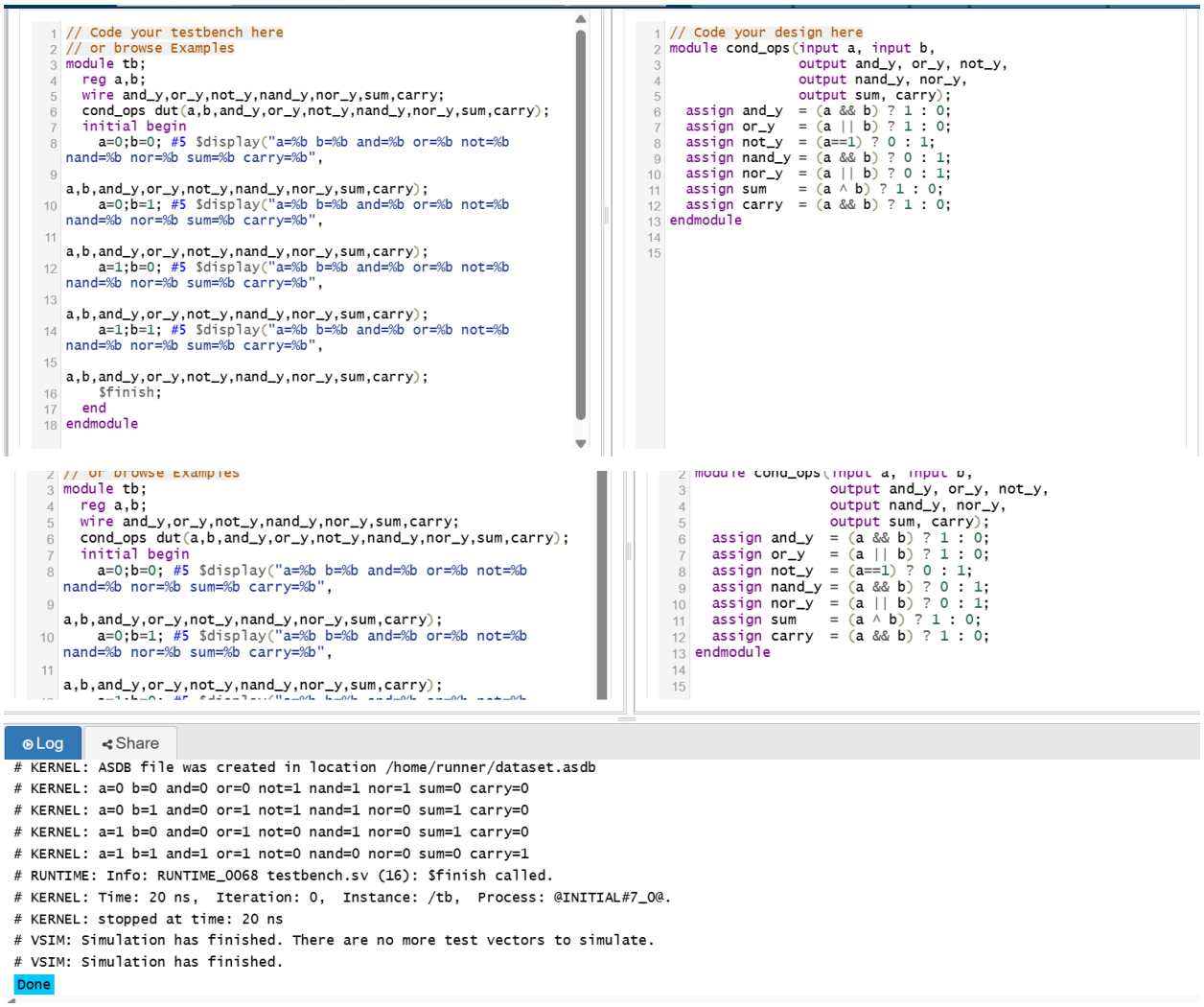
1) Implement the following using conditional operators

A) And gate B) OR gate C) Not gate D) Nand gate E) Nor gate F) Half Adder



```
// Code your testbench here
// or browse Examples
module tb;
  reg a,b;
  wire and_y,or_y,not_y,nand_y,nor_y,sum,carry;
  cond_ops dut(a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
  initial begin
    a=0;b=0; #5 $display("a=%b b=%b and=%b or=%b not=%b nand=%b nor=%b sum=%b carry=%b",
a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
    a=0;b=1; #5 $display("a=%b b=%b and=%b or=%b not=%b nand=%b nor=%b sum=%b carry=%b",
a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
    a=1;b=0; #5 $display("a=%b b=%b and=%b or=%b not=%b nand=%b nor=%b sum=%b carry=%b",
a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
    a=1;b=1; #5 $display("a=%b b=%b and=%b or=%b not=%b nand=%b nor=%b sum=%b carry=%b",
a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
    $finish;
  end
endmodule
```

```
// Code your design here
module cond_ops(input a, input b,
                output and_y, or_y, not_y,
                output nand_y, nor_y,
                output sum, carry);
  assign and_y  = (a && b) ? 1 : 0;
  assign or_y   = (a || b) ? 1 : 0;
  assign not_y  = (a==1) ? 0 : 1;
  assign nand_y = (a && b) ? 0 : 1;
  assign nor_y  = (a || b) ? 0 : 1;
  assign sum    = (a ^ b) ? 1 : 0;
  assign carry  = (a && b) ? 1 : 0;
endmodule
```

Log | Share

```
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: a=0 b=0 and=0 or=0 not=1 nand=1 nor=1 sum=0 carry=0
# KERNEL: a=0 b=1 and=0 or=1 not=1 nand=1 nor=0 sum=1 carry=0
# KERNEL: a=1 b=0 and=0 or=1 not=0 nand=1 nor=0 sum=1 carry=0
# KERNEL: a=1 b=1 and=1 or=1 not=0 nand=0 nor=0 sum=0 carry=1
# RUNTIME: Info: RUNTIME_0068 testbench.sv (16): $finish called.
# KERNEL: Time: 20 ns,  Iteration: 0,  Instance: /tb,  Process: @INITIAL#7_0@.
# KERNEL: stopped at time: 20 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Done
```

2) Implement the following using Bitwise operator and write Verilog code for the same

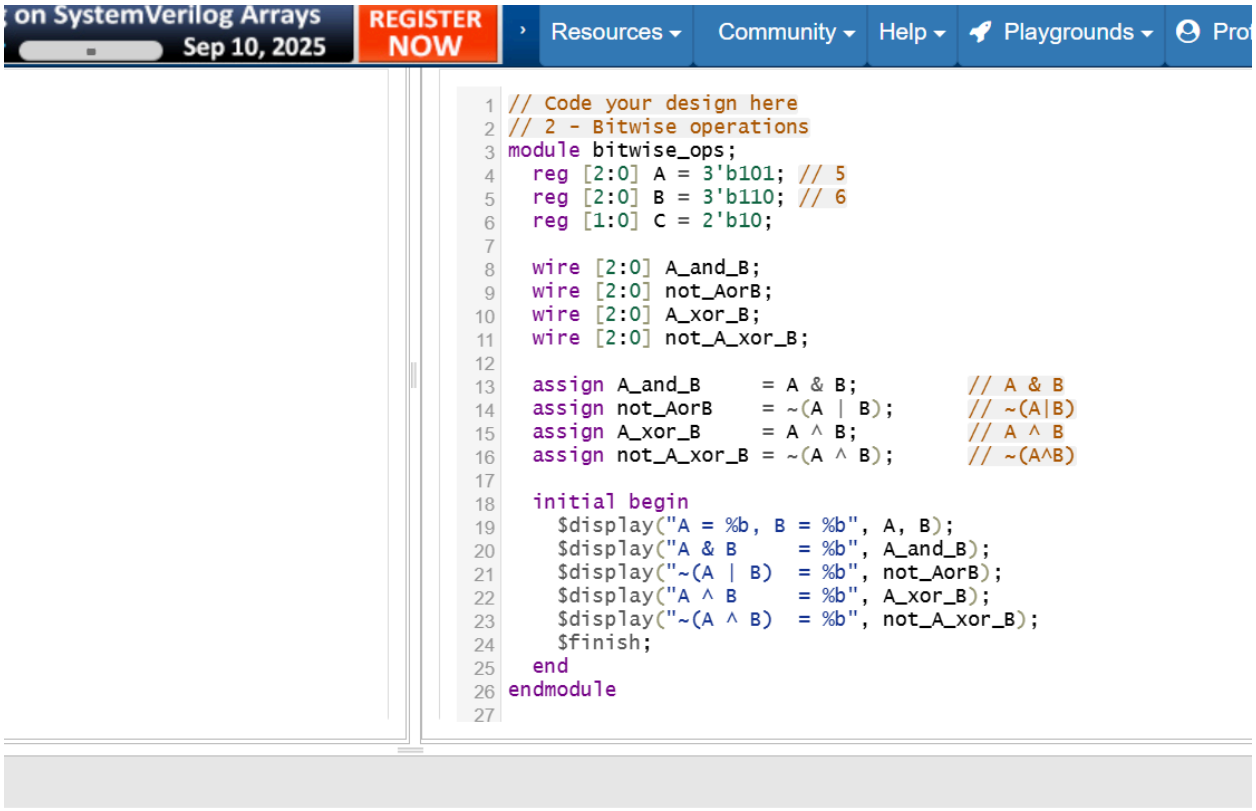
A=3'b101,B=3'b110,C=2'b10

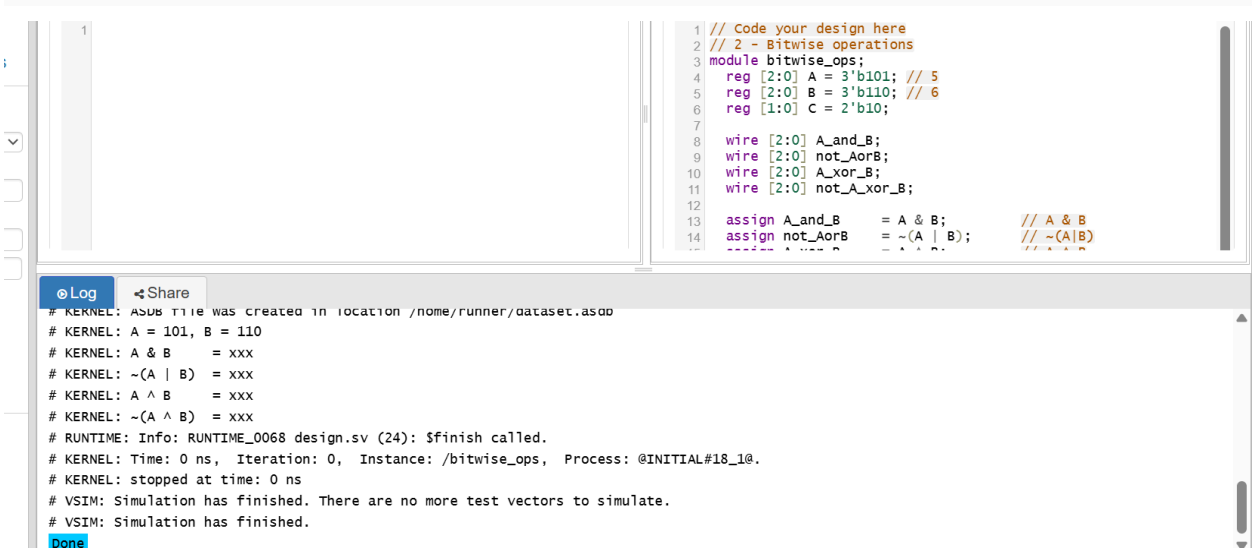
A) A&B=?

B) ~(A|B)=?

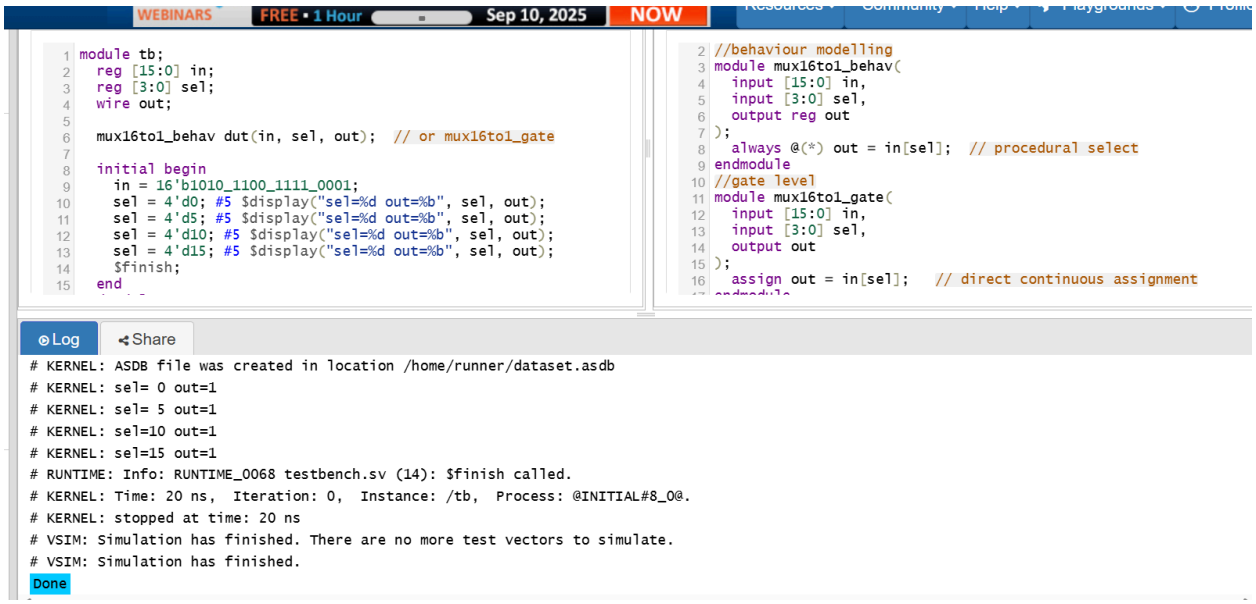
C) A^B=?

D) ~(A^B)=?





3) Write verilog code for Mux 16*1 using Gate level and Behavioral modelling.



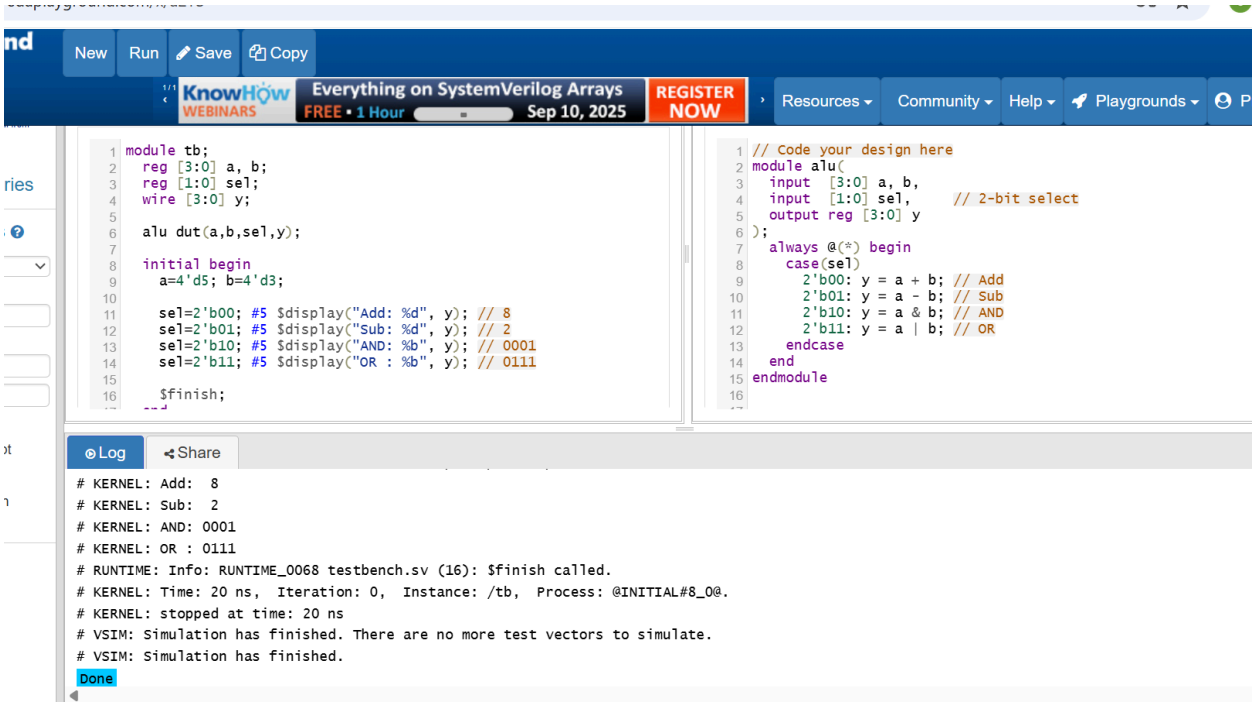
```
module tb;
  reg [15:0] in;
  reg [3:0] sel;
  wire out;

  mux16to1_behav dut(in, sel, out);  // or mux16to1_gate

  initial begin
    in = 16'b1010_1100_1111_0001;
    sel = 4'd0; #5 $display("sel=%d out=%b", sel, out);
    sel = 4'd5; #5 $display("sel=%d out=%b", sel, out);
    sel = 4'd10; #5 $display("sel=%d out=%b", sel, out);
    sel = 4'd15; #5 $display("sel=%d out=%b", sel, out);
    $finish;
  end
```

```
//behaviour modelling
module mux16to1_behav(
  input [15:0] in,
  input [3:0] sel,
  output reg out
);
  always @(*) out = in[sel];  // procedural select
endmodule
//gate level
module mux16to1_gate(
  input [15:0] in,
  input [3:0] sel,
  output out
);
  assign out = in[sel];   // direct continuous assignment
```

Log | Share

```
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: sel= 0 out=1
# KERNEL: sel= 5 out=1
# KERNEL: sel=10 out=1
# KERNEL: sel=15 out=1
# RUNTIME: Info: RUNTIME_0068 testbench.sv (14): $finish called.
# KERNEL: Time: 20 ns,  Iteration: 0,  Instance: /tb,  Process: @INITIAL#8_0@.
# KERNEL: stopped at time: 20 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Done
```

4) Write Verilog code for ALU



```
module tb;
  reg [3:0] a, b;
  reg [1:0] sel;
  wire [3:0] y;

  alu dut(a,b,sel,y);

  initial begin
    a=4'd5; b=4'd3;

    sel=2'b00; #5 $display("Add: %d", y); // 8
    sel=2'b01; #5 $display("Sub: %d", y); // 2
    sel=2'b10; #5 $display("AND: %b", y); // 0001
    sel=2'b11; #5 $display("OR : %b", y); // 0111

    $finish;
```

```
// Code your design here
module alu(
  input  [3:0] a, b,
  input  [1:0] sel,     // 2-bit select
  output reg [3:0] y
);
  always @(*) begin
    case(sel)
      2'b00: y = a + b; // Add
      2'b01: y = a - b; // Sub
      2'b10: y = a & b; // AND
      2'b11: y = a | b; // OR
    endcase
  end
endmodule
```

Log | Share

```
# KERNEL: Add:  8
# KERNEL: Sub:  2
# KERNEL: AND: 0001
# KERNEL: OR : 0111
# RUNTIME: Info: RUNTIME_0068 testbench.sv (16): $finish called.
# KERNEL: Time: 20 ns,  Iteration: 0,  Instance: /tb,  Process: @INITIAL#8_0@.
# KERNEL: stopped at time: 20 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Done
```

5) Write Verilog code for Decoder 4*1 using any levels of modelling.

```
module tb;
  reg [1:0] s; wire [3:0] y;
  decoder4to1 d(s,y);
  initial begin
    s=0; #5 $display("s=%b y=%b",s,y);
    s=1; #5 $display("s=%b y=%b",s,y);
    s=2; #5 $display("s=%b y=%b",s,y);
    s=3; #5 $display("s=%b y=%b",s,y);
  end
endmodule
```

```
// Code your design here
module decoder4to1(input [1:0] s, output reg [3:0] y);
  always @(*) begin
    y=4'b0000;
    case(s)
      2'b00: y[0]=1;
      2'b01: y[1]=1;
      2'b10: y[2]=1;
      2'b11: y[3]=1;
    endcase
  end
endmodule
```

Log | Share

```
KERNEL: Kernel process initialization done.
Allocation: Simulator allocated 4666 kB (elbread=427 elab2=4105 kernel=134 sdf=0)
KERNEL: ASDB file was created in location /home/runner/dataset.asdb
KERNEL: s=00 y=0001
KERNEL: s=01 y=0010
KERNEL: s=10 y=0100
KERNEL: s=11 y=1000
KERNEL: Simulation has finished. There are no more test vectors to simulate.
VSIM: Simulation has finished.
Done
```

6) Write Verilog code for Mux 8*1 using different loops.

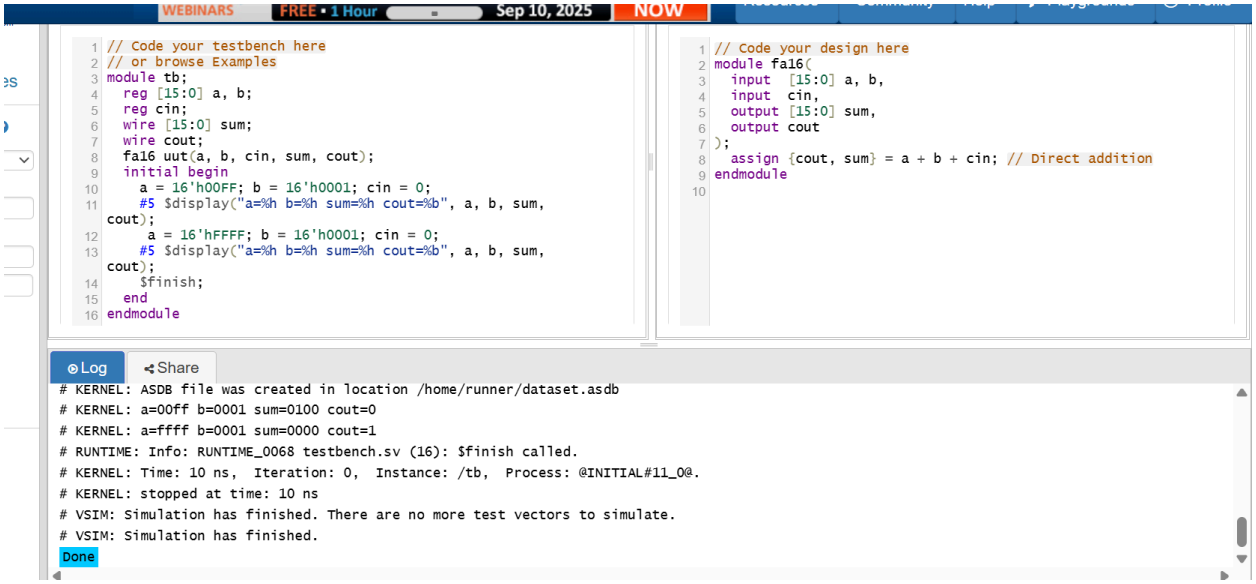

```
// Code your testbench here
// or browse Examples
module tb;
  reg [7:0] d; reg [2:0] s; wire y;
  mux8 m(d,s,y);
  initial begin
    d=8'b10101010;
    for(s=0;s<8;s=s+1) begin #5 $display("s=%d y=%b",s,y);
end
  end
endmodule
```

```
// Code your design here
module mux8(input [7:0] d, input [2:0] s, output reg y);
  integer i;
  always @(*) begin
    y=0;
    for(i=0;i<8;i=i+1)
      if(s==i) y=d[i];
  end
endmodule
```

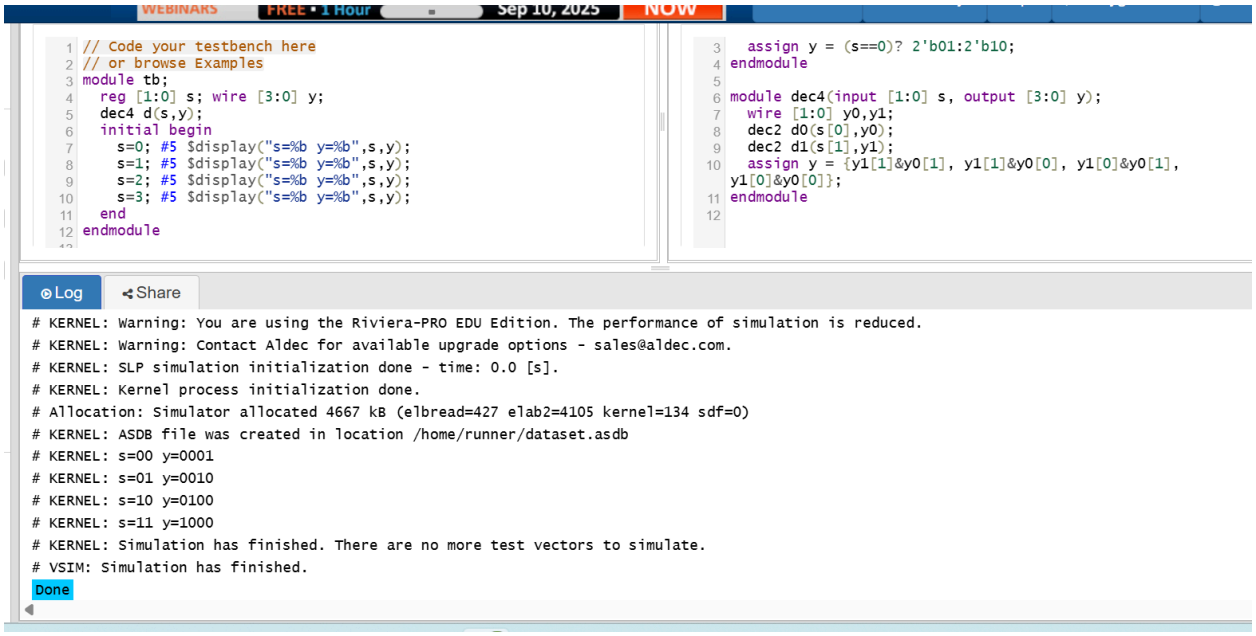
Log | Share

```
# Allocation: Simulator allocated 4665 kB (elbread=427 elab2=4104 kernel=134 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: s=0 y=0
# KERNEL: s=1 y=1
# KERNEL: s=2 y=0
# KERNEL: s=3 y=1
# KERNEL: s=4 y=0
# KERNEL: s=5 y=1
# KERNEL: s=6 y=0
# KERNEL: s=7 y=1
# KERNEL: s=0 y=0
# KERNEL: s=1 y=1
```

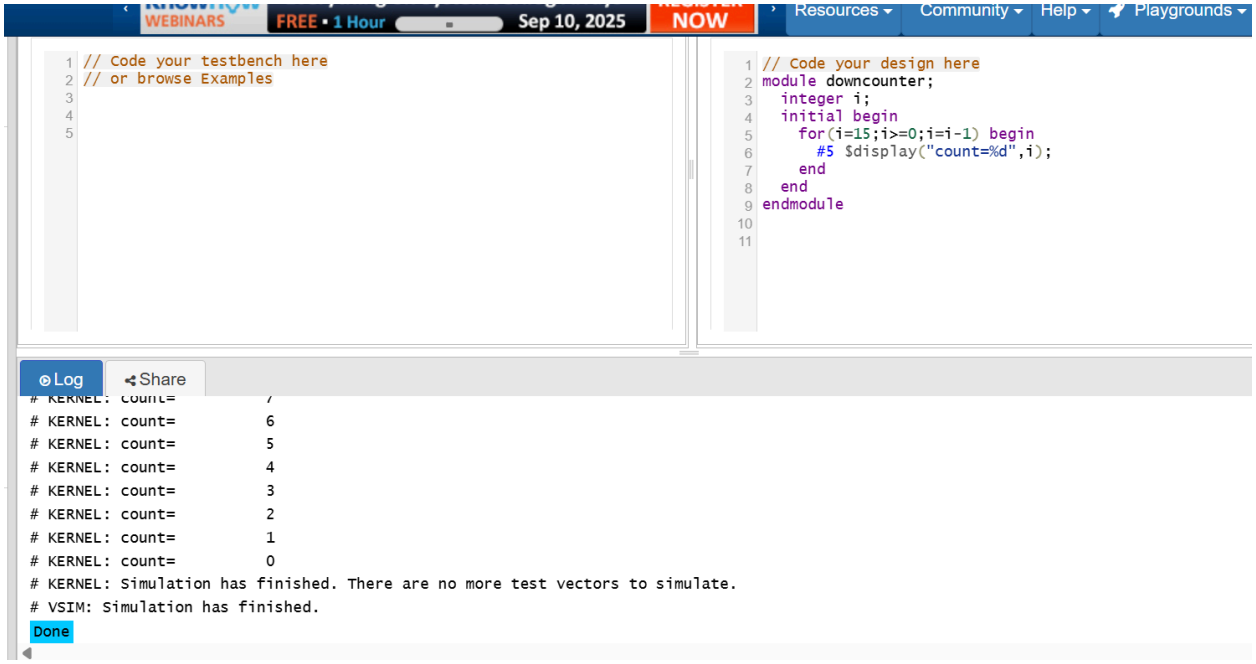
7) Write Verilog code for full adder 16-bit using full adder 4-bit using any one of the levels of modelling



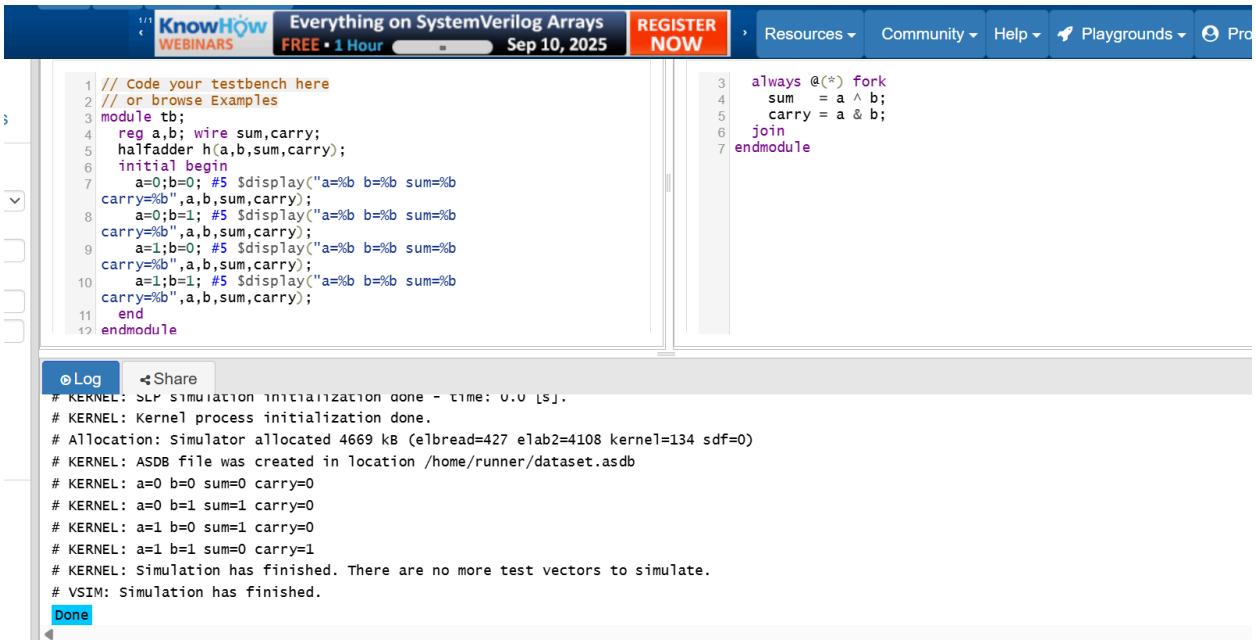
8) Write Verilog code for decoder 4*1 using decoder 2*1 using any levels of modelling



9) Write Verilog code for downcounter using loops



10) Write Verilog code for Half Adder using fork join.



SAHMA.M

REG NO:311123106055
ECE